Heap · Priority Queue · Hash Table

초보자를 위한 자료구조 학습 노트 | 개념 · 예시 · Python 코드 · 확인 문제

핵심 요약: Heap은 최솟값/최댓값을 빠르게 관리하고, Priority Queue는 중요한 일을 먼저 처리하며, Hash Table은 키를 이용해 데이터를 빠르게 찾습니다.

1. Heap(힙)

힙은 가장 크거나 작은 값을 빠르게 확인할 수 있도록 데이터를 관리하는 자료구조입니다. 예를 들어 시험 점수 [70, 90, 50, 100, 80]에서 최고 점수를 반복해서 꺼내야 할 때 유용합니다.

최대 힙과 최소 힙

최대 힙 (Max Heap)	최소 힙 (Min Heap)
부모 값 ≥ 자식 값	부모 값 ≤ 자식 값
맨 위(루트)에 최댓값	맨 위(루트)에 최솟값
예: 루트 100 → 자식 90, 80	예: 루트 10 → 자식 30, 20

힙은 완전 이진 트리입니다. 각 노드에는 최대 두 자식이 있고, 마지막 층은 왼쪽부터 채워집니다. 다만 모든 원소가 정렬된 것은 아닙니다. 부모와 자식의 크기 관계만 지키면 됩니다.

리스트에서 인덱스 i의 왼쪽 자식은 2*i+1, 오른쪽 자식은 2*i+2, 부모는 (i-1)//2입니다.

Python: heapq (기본은 최소 힙)

```
import heapq

heap = []
heapq.heappush(heap, 30)
heapq.heappush(heap, 10)
heapq.heappush(heap, 20)

print(heapq.heappop(heap))  # 10
print(heapq.heappop(heap))  # 20
print(heapq.heappop(heap))  # 30
```

heappush(heap, 값)은 삽입, heappop(heap)은 최솟값을 꺼내는 함수입니다. 숫자를 음수로 넣고 꺼낼 때 다시 부호를 바꾸면 최대 힙처럼 활용할 수 있습니다.

```
heap = []
for num in [30, 10, 20]:
    heapq.heappush(heap, -num)
print(-heapq.heappop(heap))  # 30
```

2. Priority Queue(우선순위 큐)

일반 큐는 먼저 들어온 데이터가 먼저 나가는 FIFO 방식입니다. 하지만 응급실에서는 먼저 도착한 환자보다 위급한 환자를 먼저 치료할 수 있습니다. 이런 처리 방식을 우선순위 큐라고 합니다.

일반 큐	우선순위 큐
도착 순서: A → B → C	긴급도 순서: B(1) → C(2) → A(3)
먼저 들어온 데이터가 먼저 나감	우선순위가 높은 데이터가 먼저 나감

힙과 우선순위 큐의 차이: 우선순위 큐는 어떤 순서로 처리할지 정한 자료구조이고, 힙은 이를 효율적으로 구현할 때 자주 사용하는 자료구조입니다.

Python: 환자 우선순위 처리

```
import heapq
patients = []
heapq.heappush(patients, (3, "철수"))
heapq.heappush(patients, (1, "영희"))
heapq.heappush(patients, (2, "민수"))

while patients:
    priority, name = heapq.heappop(patients)
    print(name)  # 영희 → 민수 → 철수
```

튜플 (우선순위, 이름)은 첫 번째 값부터 비교합니다. 우선순위가 같으면 이름이 비교되므로, 같은 우선순위에서 도착 순서를 보장하려면 별도 순번을 함께 저장할 수 있습니다.

3. Hash Table(해시 테이블)

해시 테이블은 키(Key)로 값을 빠르게 찾는 자료구조입니다. 학생 이름으로 점수를 검색하는 사물함 시스템처럼 생각하면 쉽습니다.

용어	뜻	예시
Key(키)	데이터를 찾는 이름표	"철수"
Value(값)	실제로 저장된 데이터	90
Hash Function(해시 함수)	키를 저장 위치와 연결하는 계산	"철수" → 특정 위치

Python에서는 dict

```
students = {"철수": 90, "영희": 85, "민수": 100}
print(students["철수"])  # 90
students["지수"] = 95   # 추가
students["철수"] = 99  # 수정
del students["영희"]   # 삭제
```

해시 테이블은 키를 통해 원하는 값에 평균적으로 빠르게 접근합니다. Python의 dict와 set은 해시 테이블 기반 자료구조입니다.

해시 충돌(Hash Collision)

서로 다른 키가 같은 저장 위치로 계산되면 해시 충돌이 발생합니다. 예를 들어 위치를 숫자 % 5로 정하면 12 % 5 = 2이고 17 % 5 = 2여서 둘 다 위치 2로 향합니다.

대표적인 해결법은 체이닝(Chaining): 같은 위치에 여러 데이터를 연결하여 보관하기, 개방 주소법(Open Addressing): 다른 빈 위치를 찾아 저장하기입니다.

4. 시간 복잡도 비교

시간 복잡도는 데이터 개수가 늘어날 때 작업량이 얼마나 증가하는지를 나타냅니다. N은 데이터의 개수입니다.

연산	Heap	Hash Table
추가	O(log N)	평균 O(1)
삭제	루트 삭제 O(log N)	키로 삭제 평균 O(1)
특정 값 찾기	일반적으로 O(N)	키로 검색 평균 O(1)
최솟값 확인	최소 힙 O(1)	일반적으로 O(N)

O(1): 데이터 개수와 무관한 일정한 작업량 / O(log N): 천천히 증가 / O(N): 데이터 개수에 비례. 해시 테이블은 충돌 등의 이유로 최악의 경우 O(N)이 될 수 있습니다.

5. 세 개념 비교

구분	Heap	Priority Queue	Hash Table
목적	최댓값·최솟값 관리	우선순위대로 처리	키로 데이터 검색
핵심	부모·자식 대소 관계	중요도 순서	해시 함수
Python	heapq	heapq / PriorityQueue	dict / set
사례	최솟값 반복 추출	작업 스케줄링	회원 정보 검색

6. 확인 문제

1) 최소 힙의 맨 위에는 어떤 값이 있을까요?

답: _____

2) 우선순위 큐에서는 무엇을 기준으로 먼저 처리할까요?

답: _____

3) Python에서 해시 테이블 기반 자료구조는 무엇일까요?

답: _____

4) 서로 다른 키가 같은 저장 위치로 계산되면 무엇이라고 부를까요?

답: _____

정답: 1) 최솟값 2) 우선순위 3) dict(또는 set) 4) 해시 충돌

한 문장씩 암기하기

Heap: 최댓값 또는 최솟값을 빠르게 꺼내는 구조.
Priority Queue: 우선순위가 높은 데이터를 먼저 처리하는 구조.
Hash Table: 키를 이용해 원하는 데이터를 빠르게 찾는 구조.